

JCMS AI/LLM Integration Blueprint

Senior AI Architect Perspective

1. Recommended AI Features & Use Cases

1.1 AI Petition Categorization & Data Extraction

Goal: Ingest a filed petition PDF and auto-extract structured fields: Case Title, Petitioner/Respondent names, Case Type (Civil/Criminal/Constitutional/Family/etc.), key dates (incident date, filing date, statutory deadlines), and relief sought.

Pipeline:

1. **OCR / text extraction** — `pdfplumber`/`PyMuPDF` for digitally-native PDFs; **Tesseract OCR** (or a cloud OCR like AWS Textract / Google Document AI) as fallback for scanned/handwritten filings.
2. **Structured extraction** — pass extracted text to an LLM with **function calling / structured output** (OpenAI `response_format: json_schema`, or Gemini's structured output mode) so the model returns a strict JSON object matching your `Case`/`Party` schema fields — not free text you have to regex-parse.
3. **Validation layer** — run extracted dates/names through deterministic checks (regex for date formats, fuzzy-match party names against existing `User`/`Party` records) before writing to DB.
4. **Human-in-the-loop confirm** — registrar sees a pre-filled form with AI-suggested values highlighted; must confirm/edit before it's committed. This is important both practically (legal filings can't silently trust AI) and as an interview talking point (you understand AI needs a human checkpoint in high-stakes domains).

1.2 Intelligent Priority Score Estimator

Goal: Read the petition narrative and suggest an urgency signal to feed into the weighted priority algorithm from the core blueprint — not replace it.

Approach:

- Treat this as a **classifier + rationale generator**, not a black-box number. Prompt the LLM to output: `{ urgency_class: "high|medium|low", rationale: string, flagged_factors: ["irreparable harm", "minor involved", "custodial detention", ...] }`.
- Map `urgency_class` + `flagged_factors` to a **bounded weight contribution** (e.g., ± 15 points) inside your existing deterministic `PriorityFactor` scoring — the AI never directly sets the final score. This keeps the algorithm auditable and explainable (critical for a judicial system) while still showcasing applied AI.

- Log every AI suggestion + the human-adjusted final score as a training/eval dataset for later fine-tuning or prompt iteration.
- Consider a lightweight **local classifier** (e.g., a fine-tuned small model or even a logistic regression on TF-IDF features) as a fast first-pass filter, with the LLM only called for borderline/ambiguous cases — good for demonstrating cost-conscious architecture thinking.

1.3 Smart Case Summarization (RAG)

Goal: "Summarize File" button generates a 1-page executive summary across all uploaded evidence, orders, and hearing history for a case.

RAG pipeline:

1. **Chunking** — split each document (petition, evidence, prior orders, hearing minutes) into ~500-1000 token chunks with overlap, tagged with metadata (`case_id`, `doc_type`, `date`, `page`).
2. **Embedding** — embed chunks with an embedding model (OpenAI `text-embedding-3-large`, Gemini `text-embedding-004`, or a local model like `bge-large-en` via `sentence-transformers` if you want an all-open-source stack).
3. **Vector store** — store embeddings scoped by `case_id` (see Section 2.2 for DB choice).
4. **Retrieval** — on "Summarize File" click, retrieve top-K relevant chunks (filtered by `case_id`, optionally re-ranked with a cross-encoder for precision).
5. **Generation** — feed retrieved chunks + a structured prompt ("You are assisting a judge; produce a neutral 1-page summary organized under Parties, Timeline, Key Evidence, Prior Orders, Open Issues") to the LLM (GPT-4-class or Gemini 1.5/2.x for long context).
6. **Citations** — have the model return which source chunk/page backs each summary claim, and render those as clickable references back to the original document — this is what separates a toy RAG demo from a production-grade one, and it's a major resume differentiator.

1.4 AI Stenographer / Transcript Summarizer

Goal: Convert raw hearing audio/text into a structured draft court order.

Pipeline:

1. **Speech-to-text** — Whisper (OpenAI API or self-hosted `whisper.cpp`/`faster-whisper` for an open-source angle) with speaker diarization (e.g., `pyannote.audio`) to separate judge/advocate/witness speech.
2. **Structuring** — pass the diarized transcript to an LLM prompted to output a **draft order template**: case number, appearances, submissions summary, judge's directions, next hearing date — again as structured JSON/sections, not free prose, so it maps cleanly onto your `Hearing.order_summary` field.

3. **Draft-only, never auto-published** — the draft is explicitly marked `AI-generated draft – pending judicial review` and only becomes an official order after the judge edits/approves it. State this constraint clearly in your README — it shows judgment about where AI belongs in a legal workflow, which recruiters/interviewers will specifically probe.

2. Technical AI Architecture

2.1 Model/Tool Selection Per Feature

Feature	Suggested Model(s)	Supporting Tools
OCR / doc text extraction	Tesseract OCR (open-source) or Google Document AI / AWS Textract (better accuracy on scans)	<code>pdfplumber</code> , <code>pdf2image</code>
Structured extraction (categorization)	GPT-4o / GPT-4.1 with structured outputs, or Gemini 1.5/2.x with JSON mode	LangChain <code>with_structured_output</code> , or raw API + Pydantic/Zod schema validation
Priority urgency classification	Small/cheap model first pass (GPT-4o-mini, Gemini Flash) → escalate to a larger model for ambiguous cases	Simple rules engine as a pre-filter
RAG summarization	Long-context model: GPT-4.1, Gemini 1.5/2.x Pro (huge context windows are genuinely useful here), or Llama 3.1/3.3 self-hosted via vLLM/Ollama for a fully open-source stack story	LangChain or LlamaIndex for retrieval orchestration
Embeddings	<code>text-embedding-3-large</code> (OpenAI), <code>text-embedding-004</code> (Gemini), or <code>bge-large-en-v1.5</code> / <code>nomic-embed-text</code> (open-source, run locally)	<code>sentence-transformers</code>
Speech-to-text	Whisper API or self-hosted <code>faster-whisper</code>	<code>pyannote.audio</code> for diarization

Framework choice: **LangChain** or **LlamaIndex** both work; LlamaIndex tends to be more RAG-focused and slightly less boilerplate for document indexing, while LangChain has a broader ecosystem (agents, tool-calling chains) if you plan to expand beyond RAG later (e.g., an agent that cross-references case law). Either is a reasonable resume line — pick one and go deep rather than mixing both.

Local LLM option (Llama 3): If you want to demonstrate you can run AI without depending on paid APIs (a strong signal for cost-conscious/infra-aware roles), self-host Llama 3.1/3.3 8B or 70B via **Ollama** (simplest) or **vLLM** (production-grade, better throughput) behind an internal API that mimics the OpenAI SDK shape — this lets you swap providers with minimal code change and is worth explicitly calling out in your README as a "provider-agnostic AI layer" design decision.

2.2 Vector Database Setup for RAG

Option	When to use
PGVector (PostgreSQL extension)	Recommended for this project. You already run Postgres for the core app — adding <code>pgvector</code> avoids a second database to operate, keeps embeddings joined naturally to <code>case_id</code> / <code>document_id</code> via foreign keys, and is a great "I made a pragmatic architecture decision" talking point instead of over-engineering with a separate vector DB.
ChromaDB	Good for local dev/prototyping or if you want a lightweight, embeddable vector store with zero infra ops.
Pinecone	Use if you want to demonstrate a managed, horizontally-scalable vector search service — a stronger "cloud-native at scale" story for Phase 3, but adds an external dependency/cost.

Recommended path: Start with **PGVector** for Phase 1/2 (simplicity, single source of truth, transactional consistency between case data and embeddings), and mention Pinecone/OpenSearch-with-kNN as the documented "scale-out path" in your README for when corpus size grows — this shows you can reason about scaling without over-building prematurely.

Schema addition:

```
sql
```

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE document_chunk (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  document_id UUID REFERENCES "Document"(id),
  case_id UUID REFERENCES "Case"(id),
  chunk_text TEXT NOT NULL,
  page_number INT,
  embedding VECTOR(1536), -- match your embedding model's dimension
  created_at TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX ON document_chunk USING ivfflat (embedding vector_cosine_ops) WITH
```

Retrieval query pattern: filter by `case_id` first (cheap index scan), then order remaining rows by cosine distance to the query embedding — this two-stage filter-then-rank is worth explaining in an interview since naive vector search often skips the metadata pre-filter and ends up needlessly expensive.

2.3 Architectural Guardrails Worth Documenting

- **Cost/latency tiering:** cheap model for classification/extraction, expensive model only for generation-heavy tasks (summarization, draft orders).
- **Prompt versioning:** store prompts in version-controlled files (not inline strings scattered in code), log which prompt version produced which output — useful for debugging and shows engineering maturity.
- **Evaluation harness:** even a small set of 15–20 hand-labeled petitions to test extraction accuracy against is a strong artifact to show in a portfolio ("here's how I measured my AI feature's precision/recall," not just "it works on my demo").
- **Human review is non-negotiable everywhere AI touches judicial output** — extraction, priority suggestions, and draft orders are all reviewable/editable before they affect real workflow state. This is both the ethically correct design for this domain and the single most interview-worthy design decision in the whole AI layer.

Suggested Build Order

1. OCR + structured extraction (highest immediate value, lowest complexity)
2. PGVector + basic RAG summarization
3. Priority urgency classifier (bounded contribution to existing algorithm)
4. Speech-to-text + draft order generation (most complex, save for last)